Database Systems 1 | Dbs1-en

Summary

CONTENTS

1. Glossary 3
2. UML 4
2.1. Additional inheritance notes 5
2.2. Crow’s foot notation 5
3. Database models 5
4. ANSI model 6
5. Database design process 6
5.1. Conceptual model 7
5.2. Logical model 7
5.3. Physical model 7
6. Normal forms 7
6.1. NF1 8
6.2. NF2 8
6.3. NF3 8
6.4. BCNF 8
7. Relational algebra 8
8. (Postgre)SQL 9
8.1. Glossary 9
8.2. DDL (Data Definition Language) 9
8.2.1. Main commands 9
8.2.2. CREATE TABLE example 10
8.2.3. Data types 10
8.2.4. Contraints 11
8.2.5. ALTER TABLE 11
8.2.6. Referential integrity 12
8.2.7. Index 12
8.2.8. Schema 14
8.2.9. Permissions 14
8.3. DML (Data Manipulation Language) 14
8.3.1. Syntax excerpt 14
8.3.2. INSERT 14
8.3.3. UPDATE 15
8.3.4. DELETE 15
8.3.5. SELECT 15
8.4. Views 19
8.4.1. Common table expressions 19
8.5. Functions 20
8.6. DCL (Data Control Language) 20
8.6.1. User management 20
8.6.2. Permissions 20

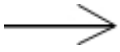
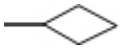
8.6.3. Groups 21
8.6.4. Read-only user 21
8.6.5. RBAC (Role-Based Access Control) 21
8.6.6. RLS (Row-Level Security) 21
8.7. Prepared statements 22
8.8. Transactions 22
8.8.1. Transaction isolation 23
8.8.2. Savepoints 24
8.8.3. Serializability 24
8.9. Backup and Recovery 24

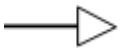
1. GLOSSARY

German term	*English term*	*Definition*
Entität	Entity	A uniquely identifiable object or concept, e.g. a person or a product.
Entitätsmenge	Entity set	A group of similar entities characterized by shared properties.
Kardinalität von Beziehungen	Relationship cardinality	Indicates how many entities participate in a relationship (e.g. 1:1, 1:n, n:m).
Kardinalität von Attributen	Attribute cardinality	Refers to the number of values an attribute can take.
Basisdatentyp	Base data type	The basic data types in a database, such as Integer, String, or Boolean.
Mehrwertiges Attribut	Multivalued attribute	An attribute that can store multiple values for a single entity object, e.g. phone numbers.
Schwache Entity Menge	Weak entity set	A set of entities that cannot exist without an associated strong entity.
Spezialisierung	Specialization	The process of deriving a more specific entity from a general entity.
Surrogate Key	Surrogate Key	An artificial key used to identify an entity without using natural properties.
Zusammengesetztes Attribut	Composite attribute	An attribute consisting of several subattributes, e.g. an address (street, city, postal code).
Zweitschlüssel	Secondary key	An attribute or combination of attributes that can uniquely identify an entity without being the primary key.
Datenbank	Database	An organized collection of data managed in a DBMS.
System-Katalog	System catalog	An internal repository containing metadata about database objects, e.g. tables and schemas.
Datenbankschema	Database schema	The logical structure of a database describing the organization of data and its relationships.
Datenbasis	Data store	The physical storage location where the database's data is actually stored.

German term	English term	Definition
Datenunabhängigkeit	Data independence	The ability to change data in a database without requiring changes to applications.
DBMS (Datenbank-managementsystem)	DBMS (Database management system)	Software that enables the creation, management, and use of a database.
DBS (Datenbanksystem)	DBS (Database system)	A general term for a combination of a DBMS and its associated databases.
Implementierungsschema	Implementation schema	The specific structure and organization of data at the physical level within a DBMS.
Impedance-Mismatch	Impedance-Mismatch	A problem that occurs when two systems use different data models, making interaction more difficult.
Konsistenz	Consistency	The state in which database rules are upheld and the data has integrity and is free of errors.
ODBMS (Objektorientiertes Datenbankmanagementsystem)	ODBMS (Object-oriented database management system)	A DBMS that uses object-oriented programming concepts for data management.
ORDBMS (Objekt-relationales Datenbankmanagementsystem)	ORDBMS (Object-relational database management system)	A combination of object-oriented and relational principles in database management.
Persistenz	Persistence	The ability of data to exist beyond a program's runtime.
RDBMS (Relationales Datenbankmanagementsystem)	RDBMS (Relational database management system)	A DBMS that stores data in tabular form and provides relational access.

2. UML

German term	English term	Definition
Assoziation	Association 	A relationship between two or more classes showing how objects of these classes are connected.
Aggregation	Aggregation 	A special form of association representing a "part-of" relationship in which the lifetime of the part is independent of the whole.
Komposition	Composition 	A stronger form of aggregation in which the lifetime of the part depends on the whole; deleting the whole also deletes the part.

German term	English term	Definition
Vererbung	Inheritance 	A mechanism through which a class can inherit properties and behavior (methods) from another class, promoting reusability.
Multiplizität 1 | 0..1 | 1..*	Multiplicity 1 | 0..1 | 1..*	Indicates how many objects of one class can be related to an object of another class (e.g. 1:1, 1:n, n:m).
Notiz	Note	An annotation or comment within a UML diagram providing additional information or explanations about elements or relationships.

2.1. ADDITIONAL INHERITANCE NOTES

German term	English term	Definition
Disjoint	Disjoint	An object is an instance of exactly one subclass.
Overlapping	Overlapping	An object can be an instance of multiple overlapping subclasses; an object can change its subclass membership during its lifetime.
Complete	Complete	All subclasses are defined
Incomplete	Incomplete	Additional subclasses are allowed

2.2. CROW'S FOOT NOTATION

One 

One (mandatory) 

Zero or one 

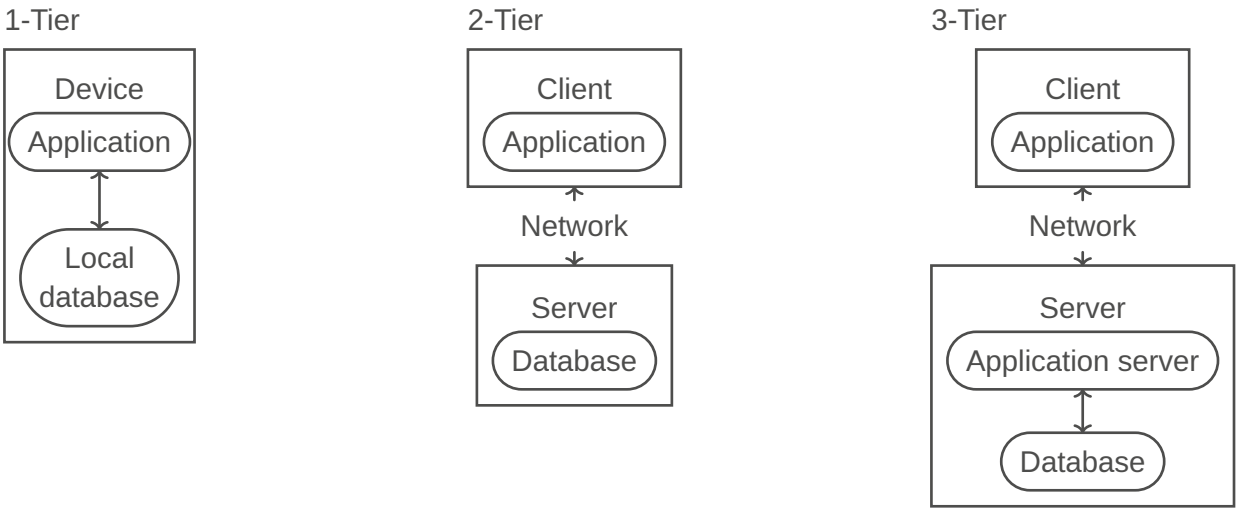
Many 

Many (at least 1) 

Zero or many 

3. DATABASE MODELS

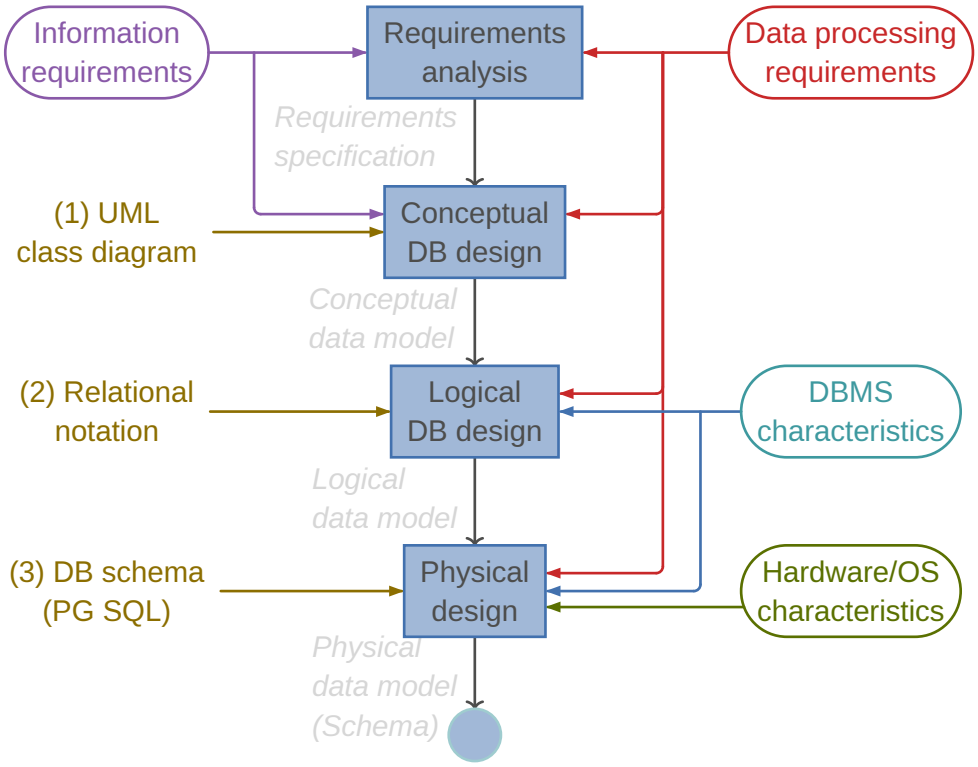
German term	English term	Definition
Hierarchisch	Hierarchical	A database model that arranges data in a tree-like structure in which each entity has one parent node and can have several child nodes.
Netzwerk	Network	A database model offering a more flexible structure than the hierarchical model by allowing multiple paths between entities.
Objektorientiert	Object-oriented	A database model that applies object-oriented programming principles to store and manage data and its behavior as objects.
Objektrelational	Object-relational	Combines object-oriented and relational principles to support complex data structures and extends relational modeling with objects.
Relational	Relational	A widely used database model that stores data in tables (relations) and manages relationships between these tables using keys.



4. ANSI MODEL

German term	*English term*	*Definition*
Logische Ebene	Logical level	Logical structure of the data, defined by the logical schema «underlying model» (DBMS access to data on the storage medium)
Interne Ebene	Internal level	Storage structures, defined by the internal schema (relationships between data, tables, etc.)
Externe Ebene	External level	A user class's view of a subset of the DB, defined by an external schema (data the user can access)
Mapping	Mapping	A more or less complex mapping is required between the levels

5. DATABASE DESIGN PROCESS



5.1. CONCEPTUAL MODEL

In this phase, an abstract view of the database is created to capture user requirements. Entities, attributes, and their relationships are defined without considering technical details.

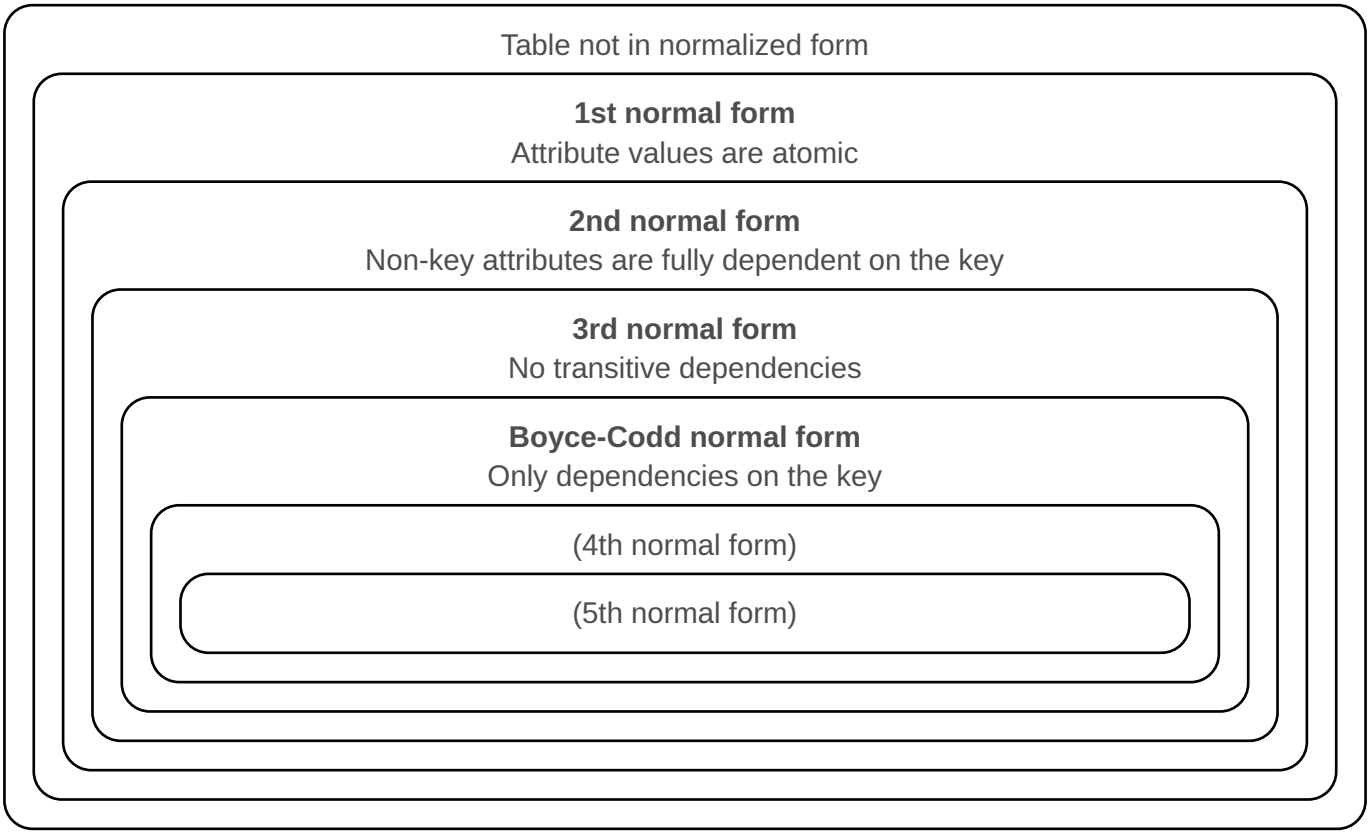
5.2. LOGICAL MODEL

Here, the conceptual model is translated into a specific database structure without considering physical aspects. This defines how data is organized (e.g. in tables) and which integrity rules apply.

5.3. PHYSICAL MODEL

In this phase, specific implementation details are defined, including the storage structure and indexing. The model is optimized to improve performance, efficiency, and data access.

6. NORMAL FORMS



German term	*English term*	*Definition*
Funktionale Abhängigkeit	Functional dependency	A relationship between two attributes in which one attribute (B) depends on another attribute (A); for each value of A, there is exactly one value of B, expressed as $A \rightarrow B$.
Atomar	Atomic	An attribute is atomic if it cannot be further divided into subattributes; it therefore contains only a single value.

German term	*English term*	*Definition*
Voll funktional abhängig	Fully functionally dependent	An attribute (B) is fully functionally dependent on another attribute (A) if it depends on A and cannot be derived from other attributes.
Teilweise funktional abhängig	Partially functionally dependent	An attribute (B) is partially functionally dependent on another attribute (A) if B depends on A, but also on part of a composite key.
Transitiv abhängig	Transitively dependent	An attribute (C) is transitively dependent on another attribute (A) if A refers to B and B refers to C, creating an indirect dependency between A and C.

6.1. NF1

– Attribute values are atomic

6.2. NF2

– Non-key attributes are fully dependent on the key
– If the PK consists of only a single attribute (i.e. it is not composite), 2nd NF is already satisfied automatically.
– A table is NOT in 2nd NF if it has a composite key and at least one non-key attribute is functionally dependent on only part of the key.

6.3. NF3

– No transitive dependencies

6.4. BCNF

– No dependencies on the key

7. RELATIONAL ALGEBRA

https://users.informatik.uni-halle.de/~brass/db23/slides/dd_relal.pdf

Term		*Meaning*	*Example*	*SQL*
Projection	π	Extracting selected columns from a relation	$\pi_{R1,R4}(R)$	`SELECT R1,R4 FROM R;`
Selection	σ	Filtering records. σ stands for $=, >, <, \neq, \leq, \geq$	$\sigma_{R1>30}(R)$	`SELECT * FROM R WHERE R1 > 30;`
Renaming	ρ	Renaming attributes	$\rho_{\mathrm{a} \leftarrow \mathrm{R}}$	`SELECT * FROM R AS a;`
Cartesian product	$\times$	Combining each row of a relation with each row of another relation `CROSS JOIN`	$(R \times S)$	`SELECT * FROM R,S;`
Join	$\bowtie$	Prerequisite: attributes r[a] and r[b] are union-compatible (type-compatible)	$R \bowtie S$	

Term		*Meaning*	*Example*	*SQL*
Theta-join	$\bowtie_\theta$	θ stands for $=, >, <, \neq, \leq, \geq$	$R[a\theta b]S$	`SELECT R.* FROM R JOIN S ON R.A :theta S.B;`
Equi-join	$\bowtie_=$	θ is $=$	$R \underset{A=B}{\bowtie} S = \rho_{A=B}(R \times S)$	`SELECT * FROM R JOIN S ON R.A = S.B;`
Natural join	$\bowtie$	Equi join without duplicate columns `ON R.A = S.A` can be omitted; columns with the same name are compared implicitly		`SELECT * FROM R NATURAL JOIN S ON R.A = S.B;`
Outer join	$\bowtie\!\!\!\!\bowtie$ $\bowtie$	Joins tuples even without a match		
Semi-join	$\ltimes$ $\rtimes$	“left outer join without join”		

8. (POSTGRE)SQL

8.1. GLOSSARY

German term	*English term*	*Definition*
Operatorbaum	Operator tree	A hierarchical structure representing the relationship between operators and operands in a query; used in the analysis of SQL expressions.
Semantische Integrität	Semantic integrity	Ensuring that data within the database is not only syntactically correct but also meaningful in content, thereby ensuring data validity.
Relation	Relation	In PostgreSQL, a relation denotes a table consisting of rows and columns, where each row corresponds to a record and each column to an attribute.
Surrogate Key	Surrogate Key	An artificially generated key used to uniquely identify records in a table and having no natural meaning; often implemented as a sequence.

8.2. DDL (DATA DEFINITION LANGUAGE)

Postgres Documentation

Defines commands for creating, deleting, and modifying table definitions, external views, constraints, and certain other database objects (index, cluster) to optimize the mapping from the logical to the internal level.

8.2.1. Main commands

Additionally, `IF (NOT) EXISTS` can be added, or `CREATE` can be replaced with `CREATE OR REPLACE`. For the `DROP` command, `CASCADE` can be added.

```
CREATE SCHEMA s;    DROP SCHEMA s;
CREATE DOMAIN d;    DROP DOMAIN d;
```

```
CREATE TABLE t;     DROP TABLE t;     ALTER TABLE t;  TRUNCATE TABLE t;
CREATE VIEW v;      DROP VIEW v;
CREATE INDEX i;     DROP INDEX i;
CREATE DATABASE db; DROP DATABASE db;
```

8.2.2. CREATE TABLE example

```
CREATE TABLE schema_name.table_name (
  counter SERIAL NOT NULL,
  important INT NOT NULL,
  field INT NOT NULL UNIQUE,
  very_long_name VARCHAR(666) DEFAULT "Based PostgreSQL user",
  age INT CHECK (age > 18),
  reference INT NOT NULL,
  someday DATE DEFAULT SYSDATE,
  another_ref INT FOREIGN KEY REFERENCES other_table.id ON UPDATE RESTRICT,
  PRIMARY KEY (counter, important),
  FOREIGN KEY (reference) REFERENCES third_table.id ON DELETE CASCADE,
  CHECK (field BETWEEN 69 AND 420)
)
```

8.2.3. Data types

Postgres Documentation

Appropriate conversions and rounding are performed implicitly.

German term	*English term*	*Definition*
INTEGER/INT	INTEGER/INT	Integer (4 bytes)
BIGINT	BIGINT	Large integer (8 bytes)
SMALLINT	SMALLINT	Small integer (2 bytes)
REAL	REAL	Single precision floating-point number (4 bytes)
NUMERIC(precision,scale)	NUMERIC(precision,scale)	Exact numeric of selectable precision
DOUBLE PRECISION	DOUBLE PRECISION	Double precision floating-point number (8 bytes)
SERIAL	SERIAL	Auto-incrementing integer (4 bytes)
BIGSERIAL	BIGSERIAL	Auto-incrementing large integer (8 bytes)
SMALLSERIAL	SMALLSERIAL	Auto-incrementing small integer (2 bytes)
CHARACTER/CHAR(size)	CHARACTER/CHAR(size)	Fixed-length, blank-padded string
VARCHAR(size)	VARCHAR(size)	Variable-length, non-blank-padded string
TEXT	TEXT	Variable-length character string
BOOLEAN	BOOLEAN	Logical Boolean (true/false)

German term	English term	Definition
DATE	DATE	Calendar date (year, month, day)
TIME	TIME	Time of day (no time zone)
TIMESTAMP	TIMESTAMP	Date and time (no time zone)
TIMESTAMP WITH TIME ZONE	TIMESTAMP WITH TIME ZONE	Date and time with time zone
INTERVAL	INTERVAL	Time interval
JSON	JSON	JSON data
UUID	UUID	Universally unique identifier
ARRAY OF base_type	ARRAY OF base_type	Array of values

8.2.3.1. Type casting

Explicit

```
CAST(5 AS float8) = 5::float8
SELECT 'ABCDEFG'::NUMERIC;                    -- error
SELECT SAFE_CAST('ABCDEFG' AS NUMERIC);       -- NULL
```

Implicit

```
SELECT 5 + 3.2;                               -- cast to 5.0 (numeric)
SELECT 'Number ' || 42;                       -- cast to '42'
SELECT true AND 1;                            -- treated as true
SELECT CURRENT_TIMESTAMP + INTERVAL '1 day'; -- cast to DATE
SELECT '100'::text + 1;                       -- cast to 100
```

8.2.4. Contraints

Postgres Documentation

German term	English term	Definition
PRIMARY KEY	PRIMARY KEY	The attribute is a primary key and therefore “UNIQUE” and “NOT NULL”
NOT NULL	NOT NULL	Attribute values must always have a defined value (the default is NULL)
UNIQUE	UNIQUE	Attribute values of all tuples in the table must be unique. UNIQUE combined with NOT NULL defines a secondary key. UNIQUE combined with NULL means that all non-NULL attribute values must be unique.
CHECK	CHECK	Allows additional constraints to be defined (see examples)
DEFAULT	DEFAULT	Sets a default value, which applies if no attribute value is specified when inserting a tuple with INSERT.
REFERENCES	REFERENCES	The attribute is a foreign key

8.2.5. ALTER TABLE

Postgres Documentation

Should be preferred for foreign keys, as recursive references can be implemented more easily this way.

```
ALTER TABLE table_name
  ADD CONSTRAINT constraint_name
  CHECK (counter < 1337);

ALTER TABLE other_table
  ADD CONSTRAINT fk_that
  FOREIGN KEY (that) REFERENCES table_name (counter);
```

8.2.6. Referential integrity

The foreign key in a dependent table must contain either a current key value from the referenced table (1:n) or NULL (1c:n). This referential integrity constraint can be violated by insert, delete, and update operations and should therefore be checked by the DBMS.

Trigger:

```
ON UPDATE
ON DELETE
```

Action:

```
CASCADE
RESTRICT
SET DEFAULT
SET NULL
```

See “CREATE TABLE example” (Page 10) .

8.2.7. Index

Stefan Keller's blog post

Use the index, luke

An index is an auxiliary data structure that provides the addresses of tuples with a given attribute value.

German term	*English term*	*Definition*
B-Baum	B-tree	A self-balancing tree used for fast data access in databases; it enables efficient insertion, deletion, and searching of records.
B+-Baum	B+-tree	All data is located only in the leaf nodes. Faster than a B-tree.
Bitmap-Index	Bitmap index	An index type that uses a bitmap for each value of an attribute to query large amounts of data efficiently and is particularly suitable for columns with few unique values.
Cluster	Cluster	A method of physically keeping data in a table together based on a specific index, improving access to multiple related records.
Füllgrad	Fill factor	The percentage of an index’s storage space actually filled with data; affects the efficiency of index searches and data storage.
Hash	Hash	An index that uses a hash function to speed up access to records; particularly useful for equality queries on very large amounts of data.

German term	*English term*	*Definition*
Heap	Heap	The default storage structure in a relational database, in which records are stored in arbitrary order without a specific index.
Indizes	Indexes	Structures that speed up searches for records in a database by providing a fast way to access data without scanning all records.
ISAM	ISAM	Indexed Sequential Access Method; an older indexing system that combines sequential and indexed storage to optimize data access.
Physische Speicherstruktur	Physical storage structure	The way data is physically stored in the database, including the use of indexes for faster data retrieval.
Überlaufseite	Overflow page	A special page used to store additional data when the original index page is full; frequently used in B-trees.
Zusammengesetzter Index	Composite index	An index based on two or more attributes to enable faster searches for combinations of these attributes rather than accessing only a single attribute.
Partieller Index	Partial index	An index applied only to part of a table based on a defined condition; enables efficient queries on frequently used subsets of the data.
Funktionaler Index	Functional index	Based on the results of a function applied to the values of one or more columns.

Two data structures:
– Data Pages (Heaps)
– Search tree

Tree access:

– Traversing the tree

– Following the leaf-node chain

– Table access (if necessary)

– Primary index vs secondary index

– Secondary index vs integrated (clustered) index

```
CREATE INDEX mytable_col_idx ON mytable (col);

SELECT id,nr,txt FROM test;
CREATE INDEX magic_idx ON test (nr,id) INCLUDE txt;

CREATE EXTENSION btree_gist;
CREATE INDEX mytable_col_idx2 ON mytable (col) USING gist (col);

DROP INDEX mytable_col_idx;

CREATE INDEX mytable_col_idx ON mytable (UPPER(col));
```

8.2.7.1. B-Tree

8.2.8. Schema

Postgres Documentation

A schema is a set of DB objects belonging to a logical database. A schema can contain the following objects: tables, views, assertions, permissions, etc.

8.2.9. Permissions

Postgres Documentation

8.3. DML (DATA MANIPULATION LANGUAGE)

Postgres Documentation

8.3.1. Syntax excerpt

```
<select> := [ 'WITH' [ 'RECURSIVE' ] <with_query> [, ...] ]
'SELECT' [ 'ALL' | 'DISTINCT' [ 'ON' ( <expression> [, ...] ) ] ]
    [ { * | <expression> [ [ 'AS' ] <output_name> ] } [, ...] ]
    [ 'FROM' <from_item> [, ...] ]
    [ 'WHERE' <condition> ]
    [ 'GROUP BY' [ 'ALL' | 'DISTINCT' ] <grouping_element> [, ...] ]
    [ 'HAVING' <condition> ]
    [ 'WINDOW' <window_name> 'AS' ( <window_definition> ) [, ...] ]
    [ { 'UNION' | 'INTERSECT' | 'EXCEPT' } [ 'ALL' | 'DISTINCT' ] <select> ]
    [ 'ORDER BY' <expression> [ 'ASC' | 'DESC' | 'USING' <operator> ] [ 'NULLS'
{ 'FIRST' | 'LAST' } ] [, ...] ]
    [ 'LIMIT' { <count> | 'ALL' } ]
    [ 'OFFSET' <start> [ 'ROW' | 'ROWS' ] ]

<from_item> := <table_name> [ * ] [ [ 'AS' ] <alias> [ ( <column_alias>
[, ...] ) ] ]
    [ 'LATERAL' ] ( <select> ) [ [ 'AS' ] <alias> [ ( <column_alias> [, ...] ) ] ]
    <with_query_name> [ [ 'AS' ] <alias> [ ( <column_alias> [, ...] ) ] ]
    <from_item> <join_type> <from_item> { 'ON' <join_condition> | 'USING'
( <join_column> [, ...] ) [ 'AS' <join_using_alias> ] }
    <from_item> 'NATURAL' <join_type> <from_item>
    <from_item> 'CROSS JOIN' <from_item>

<with_query> := <with_query_name> [ ( <column_name> [, ...] ) ] 'AS' ( <select> |
<values> | <insert> | <update> | <delete> | <merge> )
        [ 'USING' <cycle_path_col_name> ]
```

8.3.2. INSERT

Postgres Documentation

```
INSERT INTO table_name (important, field, reference) VALUES (3, 99, 7);

INSERT INTO other_table VALUES (20, "Goodbye world.") RETURNING counter;
```

8.3.2.1. INSERT ... SELECT

```
INSERT INTO combined_table
  SELECT t.age, t.very_long_name FROM table_name AS t
  INNER JOIN other_table AS o ON t.reference = o.id;
```

8.3.3. UPDATE

Postgres Documentation

The UPDATE command can be used to modify existing tuples in the database.

```
UPDATE table_name
  SET field = field * 2
  WHERE field IN (71,73,74);
```

8.3.4. DELETE

Postgres Documentation

```
DELETE FROM table_name
  WHERE field > 300;
```

8.3.5. SELECT

Postgres Documentation

```
SELECT field FROM table_name;

SELECT important, CONCAT(very_long_name, age) AS personal_info
  FROM table_name
  WHERE counter > 21
  GROUP BY personal_info
  ORDER BY age, counter DESC
  LIMIT 77;
```

8.3.5.1. Predicates (WHERE)

Postgres Documentation

```
BETWEEN ... AND ...
IN (..., ...)
LIKE '___%'          -- 3 chars or more
LIKE '%asd'          -- ending with "asd"
AND
OR
IS (NOT) NULL
```

8.3.5.2. Aggregate functions

Postgres Documentation

```
MAX()
MIN()
AVG()
```

```
SUM()
COUNT()
```

8.3.5.3. Grouping (GROUP BY and HAVING)

Postgres Documentation

GROUP BY divides the result table into groups with identical values in the GROUP BY column. NULL values in a GROUP BY column are treated as a separate group.

The ***HAVING*** clause can only follow a GROUP BY clause. It allows the selection of rows resulting from applying the GROUP BY condition (analogous to the WHERE clause). The condition in the HAVING clause must begin with a function that must appear in the SELECT clause.

```
SELECT very_long_name, age, COUNT(*)
  FROM table_name
  GROUP BY age, very_long_name
  HAVING COUNT(*) > 2;
```

8.3.5.4. Join

Postgres Documentation , Atlassian visual examples , CodeCademy visual examples

The join operation connects tables through columns with the same data type. This allows relationships between tables (implemented through foreign keys) to be resolved.

8.3.5.4.1. CROSS JOIN

Returns the Cartesian product of the two tables, i.e. each row from the first table is combined with each row from the second table.

```
SELECT a.*, b.*
  FROM a
  CROSS JOIN b;
```

8.3.5.4.2. INNER JOIN

Returns only the rows that match in both tables.

```
SELECT a.*, b.*
  FROM a
  INNER JOIN b ON a.id = b.id;
```

8.3.5.4.3. LEFT JOIN

Returns all rows from the left table and the matching rows from the right table. If there is no match, NULL values are returned for the right table.

8.3.5.4.4. RIGHT JOIN

Like left join, but reversed.

8.3.5.4.5. FULL OUTER JOIN

Returns all rows present in either table. If there is no match, NULL values are returned for the table in which no match was found.

8.3.5.4.6. JOIN LATERAL

A special type of join that allows a subquery based on rows of the outer query to be used. This is useful for complex queries in which the results of a subquery depend on the results of the outer query.

```
SELECT a.*, b.*
  FROM table_a AS a,
    JOIN LATERAL (SELECT * FROM table_b WHERE table_b.id = a.id) AS b;
```

8.3.5.5. Set operations

Postgres Documentation

Each of these operations can be followed by `ALL` to retain duplicates.

8.3.5.5.1. UNION

Combines the results of two or more SELECT queries and returns all rows, excluding duplicates.

```
SELECT r1, r2
FROM A
UNION ALL
SELECT r1, r2
FROM B;
```

8.3.5.5.2. INTERSECT

Returns the rows common to two SELECT queries. The result contains only rows present in both queries.

8.3.5.5.3. EXCEPT (MINUS)

Returns rows from the first SELECT query that are not present in the second SELECT query.

8.3.5.6. Subqueries

Postgres Documentation

A subquery may contain only one column name or one expression and no ORDER BY clause.

Nested example:

```
SELECT name
  FROM mitarbeiter
  WHERE id IN
  (SELECT mitarbeiterNr
    FROM projektZuteilung
    WHERE projNr IN
    (SELECT projNr
      FROM projekt INNER JOIN mitarbeiter
      ON projekt.projLeiter = mitarbeiter.id
      WHERE name = 'Kropotkin, Peter'
    )
  );
```

8.3.5.6.1. =

Compares the value of a column with the result of a subquery. The subquery may return only one value.

```
SELECT *
  FROM mitarbeiter
  WHERE gehalt = (SELECT MAX(gehalt) FROM mitarbeiter);
```

8.3.5.6.2. IN

Checks whether the value of a column is contained in the set of results of a subquery.

```
SELECT *
  FROM bestellungen
  WHERE kundenID IN (SELECT id FROM kunden WHERE land = 'CH');
```

8.3.5.6.3. EXISTS

Checks whether a subquery returns at least one row.

```
SELECT name
  FROM mitarbeiter
  WHERE EXISTS
  (SELECT * FROM projektZuteilung WHERE mitarbeiterNr = mitarbeiter.id);
```

8.3.5.6.4. ANY

Checks whether the value of a column satisfies the condition for any value in the set of results of a subquery.

```
SELECT *
  FROM mitarbeiter
  WHERE gehalt > ANY (SELECT gehalt FROM manager);
```

8.3.5.6.5. ALL

Checks whether the value of a column satisfies the condition for all values in the set of results of a subquery.

8.3.5.7. Window functions

Postgres Documentation

Window functions are special SQL functions that perform calculations over a set of rows related to the current row.

```
SELECT
  mitarbeiter,
  gehalt,
  RANK() OVER (ORDER BY gehalt DESC) as gehaltsrang
FROM
  mitarbeitertabelle;

SELECT
  persnr,
  name,
  LAG(salaer, 1) OVER (
    PARTITION BY abtnr
  ORDER BY
    salaer DESC
  ) - salaer AS differenz
FROM
  angestellter;
```

8.3.5.7.1. Functions

Postgres Documentation

German term	*English term*	*Definition*
RANK()	RANK()	Assigns ranks while accounting for ties. Multiple equal values receive the same rank, and the next position is skipped
DENSE_RANK()	DENSE_RANK()	Like RANK, but without gaps
PERCENT_RANK()	PERCENT_RANK()	Calculates the percentage rank of a record within a window
ROW_NUMBER()	ROW_NUMBER()	Assigns a unique sequential number to each row. Even for equal values, each row receives a different number
LAG(value, offset)	LAG(value, offset)	Accesses the value of a preceding row in the window
LEAD(value, offset)	LEAD(value, offset)	Accesses the value of a following row in the window
FIRST_VALUE(value)	FIRST_VALUE(value)	Returns the first value in the defined window set. Useful for comparisons with the initial value of a partition
LAST_VALUE(value)	LAST_VALUE(value)	Returns the last value in the defined window set
NTH_VALUE(value, n)	NTH_VALUE(value, n)	Returns the value of the nth record within a window
NTILE(n)	NTILE(n)	Divides the records into n equally sized groups

8.3.5.7.2. OVER clause

German term	*English term*	*Definition*
ORDER BY	ORDER BY	Sorts the rows within the window by one or more columns. Determines the order in which calculations for window functions are performed
PARTITION BY	PARTITION BY	Divides the result set into partitions to which window functions are applied separately. Enables calculations within defined groups without changing the overall result set

8.4. VIEWS

A view is a virtual table mapped to one or more tables or views. The mapping is defined using a SELECT statement. The view's data is derived from the underlying table data only at runtime.

Views allow different users to see the data structured differently. Views can simplify the formulation of queries spanning multiple tables. Views provide effective access control by allowing columns of the underlying tables to be hidden.

```
CREATE VIEW mitarbeiter_public (id, name, tel) AS
  SELECT id, name, tel
  FROM mitarbeiter;
```

8.4.1. Common table expressions

Auxiliary query in a WITH clause (temporary tables during the statement). The query name always appears in FROM.

```
WITH queryName AS ( SELECT * FROM myTable )
SELECT * FROM queryName;

WITH tmptable(name, bezeichnung, zeitanteil) AS (
  SELECT name, bezeichnung, zeitanteil
  FROM angestellter a
  JOIN projektzuteilung pz ON pz.persnr=a.persnr
  JOIN projekt p ON p.projnr=pz.projnr
)
SELECT name AS "Mitarb.", bezeichnung AS "Projekt", zeitanteil AS "Zeit"
FROM tmptable;
```

8.4.1.1. Recursive

A recursive CTE references itself to return subsets of the data until all results have been obtained.

```
WITH RECURSIVE untergebene(persnr, name, chef) AS (
  SELECT A.persnr, A.name, A.chef FROM angestellter A
  WHERE A.chef = 1010 UNION ALL -- recursive term
    SELECT A.persnr, A.name, A.chef FROM angestellter A
  INNER JOIN untergebene B ON B.persnr = A.chef
)
SELECT * FROM untergebene ORDER BY chef, persnr;
```

8.5. FUNCTIONS

```
CREATE PROCEDURE
EXEC
```

8.6. DCL (DATA CONTROL LANGUAGE)

8.6.1. User management

Postgres Documentation Postgres Documentation

ROLE: General term for a user (***USER***) or group (***GROUP***).

```
CREATE ROLE user_name
WITH LOGIN PASSWORD 'password';

-- set current user
SET ROLE user_name;

DROP ROLE user_name;
```

8.6.2. Permissions

Postgres Documentation

8.6.2.1. System

```
GRANT INSERT ON TABLE table_name TO user_name;
REVOKE INSERT ON TABLE table_name TO user_name;
```

8.6.2.2. Object

```
ALTER ROLE user_name CREATEROLE, CREATEDB, INHERIT;
```

8.6.3. Groups

```
CREATE ROLE manager;
GRANT SELECT ON table_name TO manager;
GRANT manager TO user_name;
```

8.6.4. Read-only user

```
-- creating
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
CREATE ROLE readonlyuser WITH LOGIN ENCRYPTED PASSWORD 'readonlyuser'
NOINHERIT;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonlyuser;
-- assign permissions to read all newly tables created in the future (by
others):
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO
readonlyuser;
-- deleting
REVOKE SELECT ON ALL TABLES IN SCHEMA public FROM readonlyuser;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  REVOKE SELECT ON TABLES FROM readonlyuser;
DROP USER readonlyuser;
```

8.6.5. RBAC (Role-Based Access Control)

This is what is explained above.

8.6.6. RLS (Row-Level Security)

– Check constraint on a table per role (effectively an additional WHERE)
– (Tip: Take care with TRUNCATE, REFERENCES, and VIEWS)

```
CREATE TABLE exams (
  id SERIAL,
  student_name TEXT,
  grade DECIMAL(2,1),
  added TIMESTAMP DEFAULT current_timestamp,
  teacher_pguser VARCHAR(60) DEFAULT current_user
);

CREATE POLICY policy_teachers_see_own_exams ON exams
FOR ALL
TO PUBLIC
USING (teacher_pguser = current_user);
```

```
ALTER TABLE exams
ENABLE ROW LEVEL SECURITY;
```

8.7. PREPARED STATEMENTS

Postgres Documentation

8.8. TRANSACTIONS

German term	*English term*	*Definition*
Fault Tolerance	Fault Tolerance	In the event of a server crash, an operation can be repeated or is canceled entirely (rather than only partially executed)
Concurrency	Concurrency	Transaction isolation enables concurrent execution.
MVCC (Multi-Version Concurrency Control)	MVCC (Multi-Version Concurrency Control)	Managing multiple transactions simultaneously without conflicts
Locking	Locking	A mechanism for controlling access to data in a database, ensuring that only one transaction at a time can access or modify certain data to avoid inconsistencies.
2-Phasen-Sperrprotokoll	Two-phase locking protocol	A protocol that forces transactions to first acquire all required locks (phase 1) and then acquire no new locks (phase 2). This guarantees transaction serializability.
Lost-Update	Lost-Update	A problem that occurs when two transactions read and modify the same record. The changes made by the first transaction are lost because the second transaction overwrites them without taking them into account.
Non-Repeatable Read	Non-Repeatable Read	A situation in which a transaction reads a record and another transaction modifies the same record during that transaction. A later read in the first transaction could return a different value.
Optimistisches Lockverfahren	Optimistic locking	An approach in which transactions operate without initial locks and check at the end whether their changes are valid. If conflicts occur, changes are rolled back.
Pessimistisches Lockverfahren	Pessimistic locking	An approach in which transactions request locks immediately to ensure that other transactions cannot access or modify the same data simultaneously.
Semantische Integrität	Semantic integrity	Ensuring that data in the database is not only syntactically correct but also meaningful and correct in content, especially after transaction operations.
SLOCK (Shared Lock)	SLOCK (Shared Lock)	A lock type that allows multiple transactions to read data simultaneously but prevents writing while a transaction is in progress.
XLOCK (Exclusive Lock)	XLOCK (Exclusive Lock)	A lock type that grants one transaction exclusive read and write access to a record and prevents all other transactions from accessing it until the lock is released.

German term	*English term*	*Definition*
UNLOCK	UNLOCK	The command or action for releasing a previously acquired lock, allowing other transactions to access the locked data again.
Starvation	Starvation	A situation in which a transaction never gets the opportunity to complete its work because locks or other contention mechanisms keep blocking it.
A Atomicity	***A*** Atomicity	All or nothing
C Consistency	***C*** Consistency	A consistent state is preserved
I Isolation	***I*** Isolation	A transaction should be isolated from others
D Durability	***D*** Durability	All changes are persistent

```
BEGIN;

COMMIT;
```

```
BEGIN;

ROLLBACK;
```

8.8.1. Transaction isolation

https://pgdash.io/blog/postgres-transactions.html

```
READ UNCOMMITTED
READ COMMITTED
REPEATABLE READ
SERIALIZABLE
```

8.8.1.1. Error types

German term	*English term*	*Definition*
Dirty Read	Dirty Read	A dirty read occurs when a transaction reads data modified by another transaction that has not yet completed. This can lead to inconsistent data, as the initial changes may later be rolled back.
Fuzzy Read	Fuzzy Read	In a fuzzy read, a transaction reads data that other transactions modify during its execution. As a result, the transaction may read different values when it executes the same query multiple times.
Phantom Read	Phantom Read	A phantom read occurs when a transaction executes a query that returns new records due to changes in another transaction. This often happens when new records are inserted after the query was originally executed.

German term	English term	Definition
Serialization Anomaly	Serialization Anomaly	This anomaly occurs when the results of transactions executed in a particular order differ from those that would occur if the transactions were executed serially in a different order. This can lead to inconsistent results.

8.8.1.2. READ COMMITTED

Default isolation level in PostgreSQL.
Each query sees only data committed before the query began. Prevents “Dirty Reads”

8.8.1.3. REPEATABLE READ

Prevents “Non-Repeatable Reads”. Uses snapshot isolation. Higher memory requirements due to snapshots.

8.8.1.4. SERIALIZABLE

If the same column is UPDATEd in multiple transactions, the transactions are aborted (except the first).

8.8.2. Savepoints

Postgres Documentation

```
BEGIN;
...
SAVEPOINT savepoint1;
...
ROLLBACK TO SAVEPOINT savepoint1;
COMMIT;
```

8.8.3. Serializability

https://www.youtube.com/watch?v=01MDhIXiXIY

German term	English term	Definition
Seriell	Serial	When all transactions in a schedule are ordered
Konfliktäquivalent	Conflict-equivalent	When the order of all pairs of conflicting actions is the same in both schedules
Konfliktserialisierbar	Conflict-serializable	When a schedule is conflict-equivalent to a serial schedule

8.9. BACKUP AND RECOVERY

German term	English term	Definition
WAL (Write-Ahead-Logging)	WAL (Write-Ahead-Logging)	A logging method that ensures all changes to the database are first stored in a log before being written to the database. This enables recovery in the event of a failure
Physischer Backup	Physical backup	A backup that creates a copy of all database files (including log files). This type of backup is useful for restoring an entire database

German term	*English term*	*Definition*
Logischer Backup	Logical backup	A backup that represents the structure and contents of the database at a higher level, e.g. by exporting tables as SQL scripts. This type of backup is useful for migration between databases
Full Backup	Full Backup	A complete backup that saves all data in a database at a particular point in time. This is the most comprehensive type of backup and enables full recovery
Incremental Backup	Incremental Backup	A backup that saves only changes made since the last backup (full or incremental). This saves storage space and reduces backup time